Hannah D'Cruz
Dr. Ming Li
CS 2123
5 Nov 2025

Edit Distance

The goal of this project was to implement an algorithm using dynamic programming that could find the minimum number of changes needed to get from one string of letters to another. The specific example given was differences in gene sequences in a string of DNA, which I thought was very interesting to have a practical application to our problem.

The two parts of a dynamic programming problem are building from the bottom up, and then tracing back to find the correct solution. To start from the bottom up, I made an empty 2D list to hold all the possible combinations of letters to get from one string to another. Each cell stores the edit distance to get from that substring of the source to target sequences, as well as which operation was used to get there. The first cell is filled in manually with 0, 0 because it takes no operations to get from a blank string to another blank string. We represent a blank string with a space character instead of nothing, so that the loops count it as a possibility when editing and we do not go out of range. To avoid going out of range, we also fill in the first row and first column manually with the only possible option, either an insert or a delete operation. Then we fill in the rest of the list by choosing the minimum possible edit distance. The first case is if the source and target letters at the given index match, we can keep the letter and it does not count as an edit operation. Thus, the edit distance is the same as before that letter existing in either the source or target string, which is the cell up and to the left. If the source and target letters are the same, we can still substitute the target letter for the source letter, but it will count as an edit operation, so we add one to the upper left cell. To allow the code to ignore whichever option is not valid, based on if the letters match or not, I set the value of keep or substitute to the maximum integer value so that it will always be the greatest option and will never be chosen as the minimum. The second case is inserting the new target letter to the end of the current string, which always counts as an edit operation, so we add one to the cell to the left that has the edit distance before considering the current letter. The third case is deleting the current letter from the source string. It also always counts as an edit operation, so we add one to the cell above the current one. We compare the values calculated for the three cases and choose the minimum one. We do the comparison in the specific order of case 1, then case 2, then case 3 so that ties always pick the first smallest case.

Once we have filled in the entire table, we need to find which of those steps are actually used in the optimal solution. Because the end result needs to consider both complete strings, we start in the bottom right cell. Based on which step was taken to calculate that value (keep, substitute, insert, or delete), we move back to the adjacent cell it built off of when building up the table. We add the direction it went and the source and target letters to arrays to make displaying the answer easier.

Once the function was written, I tested it on a couple of input gene sequences. I also tested to see what would happen when we reverse the source and target inputs. The difference between the original and reversed source versus target was the new version kept the same edit distance and keep operations, but insert operations turned to delete operations and delete operations turned to insert operations. The substitution operations were still there, but the order of the letters was reversed. This makes sense because insert and delete our inverse operations, and if the letter is kept, it is the same in both strings. Overall, this project gave me a better understanding of edit distances and its applications.